

UNIVERSIDAD DE PAMPLONA

FACULTAD DE INGENIERÍAS Y ARQUITECTURA
PROGRAMA INGENIERÍA DE SISTEMAS
INGENIERÍA DE SOFTWARE I

DINAMYT ACADEMY

Plataforma Digital de Enseñanza en Artes Marciales

DOCUMENTO INTEGRAL DEL PROYECTO

AMIR DANIEL SARMIENTO AVELLANEDA — 1093295569

DOCENTE

GEOVANNY RUBIANO TORRES

VILLA DEL ROSARIO
2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO	2
1. REQUERIMIENTOS DE SOFTWARE.....	5
1.1 Documento SRS — Especificación de Requisitos de Software.....	5
1.1.1 Propósito y Alcance	5
1.1.2 Requerimientos Funcionales por Módulo	5
1.1.3 Requerimientos No Funcionales	6
1.2 Matriz de Trazabilidad	6
1.3 Modelos de Casos de Uso.....	7
1.3.1 Caso de Uso: Autenticación y Perfil	7
1.3.2 Caso de Uso: Gestión de Contenidos y Evaluaciones.....	8
1.3.3 Descripción de Caso de Uso Principal — CU-003: Realizar Evaluación.....	8
2. ARQUITECTURA DE SOFTWARE.....	9
2.1 Documento SAD — Descripción de la Arquitectura del Sistema	9
2.1.1 Vista General de la Arquitectura.....	9
2.1.2 Stack Tecnológico.....	9
2.2 Diagrama de Contenedores (Modelo C4)	9
2.3 Registro de Decisiones Arquitectónicas (ADR).....	10
ADR-001: Elección de Supabase como Backend-as-a-Service.....	10
ADR-002: PWA con Next.js sobre aplicación nativa.....	10
ADR-003: Arquitectura modular en NestJS	11
3. DISEÑO DE SOFTWARE	12
3.1 Diagrama Entidad-Relación	12
3.1.1 Descripción de Entidades Principales	12
3.2 Diagrama de Clases	14
3.3 Contratos de API (OpenAPI/Swagger).....	14
3.4 Prototipos UI/UX.....	15
3.4.1 Panel del Estudiante	15
3.4.2 Panel del Maestro — Gestión de Contenidos.....	16
3.4.3 Panel Administrativo	16
4. CONSTRUCCIÓN DE SOFTWARE	17
4.1 Código Fuente	17
4.2 Scripts de Base de Datos	17
4.2.1 Esquema Principal (Resumen).....	17
4.3 Documentación del Desarrollador (README)	18
5. PRUEBAS DE SOFTWARE	19
5.1 Plan de Pruebas	19
5.2 Casos de Prueba.....	19
5.3 Reporte de Cobertura	20

6. OPERACIONES DE INGENIERÍA DEL SOFTWARE	21
6.1 Scripts de IaC — Infraestructura como Código	21
6.2 Pipelines de CI/CD	21
6.3 Tableros de Monitoreo	22
7. MANTENIMIENTO DE SOFTWARE	23
7.1 Manual de Operaciones	23
7.2 Plan de Parches	23
7.3 Registro de Deuda Técnica	24
8. GESTIÓN DE CONFIGURACIÓN DE SOFTWARE	25
8.1 Estrategia de Ramas (Gitflow)	25
8.2 Notas de Liberación (Release Notes)	26
Versión 1.0.0 — Sprint 3 (Entrega Académica Final)	26
Versión 0.2.0 — Sprint 2	26
Versión 0.1.0 — Sprint 1	26
8.3 Inventario de Artefactos y Dependencias	26
9. GESTIÓN DE INGENIERÍA DEL SOFTWARE	27
9.1 WBS / EDT — Estructura de Desglose del Trabajo	27
9.2 Cronograma del Proyecto	28
9.3 Matriz de Riesgos	28
10. MODELOS Y MÉTODOS DE INGENIERÍA DEL SOFTWARE	29
10.1 Historias de Usuario	29
10.2 Diagramas de Dominio (DDD)	30
11. PROCESO DE INGENIERÍA DEL SOFTWARE	31
11.1 Acuerdos de Equipo — DoR y DoD	31
Definición de Listo (DoR — Definition of Ready)	31
Definición de Hecho (DoD — Definition of Done)	31
11.2 Reporte de Retrospectiva	31
Sprint 1 — Retrospectiva	31
Sprint 2 — Retrospectiva	31
12. CALIDAD DEL SOFTWARE	32
12.1 Reporte de Análisis Estático	32
12.2 Métricas de Calidad	32
13. SEGURIDAD DEL SOFTWARE	33
13.1 Modelo de Amenazas (STRIDE)	33
13.2 Reporte de Análisis SAST/DAST	34
SAST — Análisis Estático de Seguridad	34
DAST — Análisis Dinámico de Seguridad	34
14. ECONOMÍA DE INGENIERÍA DEL SOFTWARE	35
14.1 Análisis de ROI	35

14.1.1 Beneficios Identificados.....	35
14.1.2 Cálculo de ROI (Proyección a 1 Año).....	35
14.2 Presupuesto Base	36

1. REQUERIMIENTOS DE SOFTWARE

1.1 Documento SRS — Especificación de Requisitos de Software

El presente documento SRS (Software Requirements Specification) describe de manera formal los requisitos funcionales y no funcionales del sistema DINAMYT Academy, conforme al estándar IEEE 830. Constituye el acuerdo base entre el equipo de desarrollo y los stakeholders sobre el comportamiento esperado del sistema.

1.1.1 Propósito y Alcance

DINAMYT Academy es una plataforma web progresiva (PWA) destinada a la enseñanza y gestión del aprendizaje en artes marciales. Su alcance incluye los módulos de autenticación, gestión de contenidos por grado de cinturón, evaluaciones con evidencia multimedia, seguimiento del progreso del estudiante y administración global de la plataforma. Quedan fuera del alcance los pagos en línea, la transmisión en vivo y la certificación digital oficial de grados.

1.1.2 Requerimientos Funcionales por Módulo

Los requerimientos funcionales se organizan en seis módulos. A continuación se presentan los principales:

ID	Módulo	Descripción	Prioridad
RF-01	Autenticación	Registro de usuarios con nombre, correo, contraseña, fecha de nacimiento, arte marcial y grado. Hash bcrypt.	Alta
RF-02	Autenticación	Envío de OTP de 6 dígitos al correo para completar el registro. Expiración 10 minutos.	Alta
RF-03	Autenticación	Login con correo y contraseña. Retorna JWT firmado con expiración configurable (24 h por defecto).	Alta
RF-04	Autenticación	Recuperación de contraseña vía código OTP enviado al correo.	Alta
RF-05	Autenticación	Control de acceso por roles: Administrador, Maestro, Estudiante.	Alta
RF-07	Artes Marciales	El Administrador registra nuevas artes marciales con nombre, descripción y jerarquía de grados.	Alta
RF-11	Contenidos	El Maestro crea unidades de contenido por grado: título, descripción y tipo (PDF, video, imagen, texto).	Alta
RF-15	Contenidos	El Estudiante accede únicamente al contenido de su grado actual y grados previos aprobados.	Alta
RF-17	Evaluaciones	El Maestro crea evaluaciones por grado con número máximo de intentos y fecha de disponibilidad.	Alta
RF-19	Evaluaciones	Preguntas de envío de evidencia (video/imagen). Calificación manual por el Maestro.	Alta
RF-24	Progreso	Panel de progreso del Estudiante: porcentaje de contenido visto, evaluaciones y calificaciones.	Alta
RF-25	Progreso	El Maestro aprueba el avance de grado del Estudiante. El sistema actualiza el grado automáticamente.	Alta
RF-28	Administración	El Administrador gestiona usuarios: crear, asignar roles, suspender y eliminar (soft delete).	Alta

ID	Módulo	Descripción	Prioridad
RF-30	Administración	Reportes de actividad: usuarios por arte marcial, evaluaciones completadas y tasa de avance de grado.	Media

1.1.3 Requerimientos No Funcionales

ID	Categoría	Descripción	Nivel
RNF-01	Rendimiento	API REST responde en < 1000 ms bajo carga normal. Carga de páginas ≤ 3 s en 4G.	Alta
RNF-02	Disponibilidad	Disponibilidad del 99 % en horario académico. Supabase SLA 99.9 %.	Alta
RNF-04	Seguridad	Contraseñas como hash bcrypt con coste mínimo 10. Sin texto plano en logs.	Alta
RNF-05	Seguridad	JWT en cada solicitud protegida. Expiración 24 h. Transmisión en Authorization: Bearer.	Alta
RNF-06	Seguridad	RBAC en endpoints y vistas. Rutas públicas solo exponen información general.	Alta
RNF-09	Usabilidad	Interfaz funcional desde 360 px de ancho. Sin zoom requerido en pantallas de 5 pulgadas.	Alta
RNF-13	Mantenibilidad	Backend organizado en módulos por dominio (auth, usuarios, contenido, evaluaciones, progreso).	Media
RNF-17	Integridad	Integridad referencial con claves foráneas. Sin registros huérfanos.	Alta
RNF-19	Auditoría	Todas las tablas registran created_at y updated_at. Historial de evaluaciones inmutable.	Alta

1.2 Matriz de Trazabilidad

La matriz de trazabilidad conecta cada requerimiento funcional con los módulos de código y los casos de prueba correspondientes, garantizando cobertura completa y rastreabilidad bidireccional.

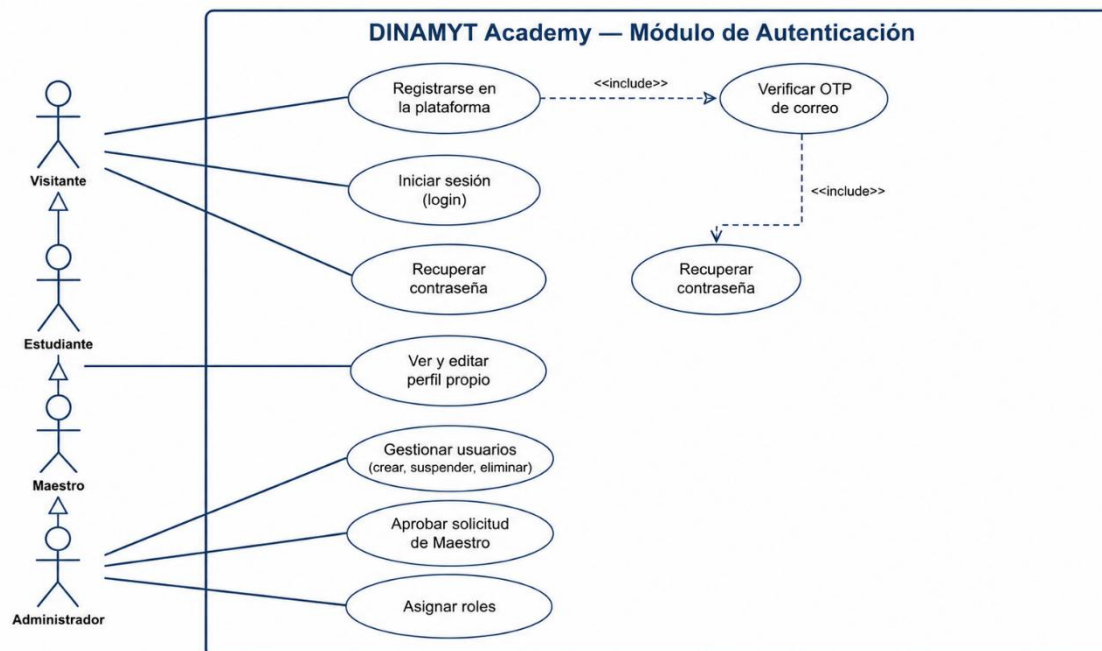
Req. ID	Módulo Código	Endpoint/Componente	Caso de Prueba
RF-01	auth.module	POST /api/auth/register	TC-AUTH-01
RF-02	auth.module	POST /api/auth/verify-otp	TC-AUTH-02
RF-03	auth.module	POST /api/auth/login	TC-AUTH-03
RF-04	auth.module	POST /api/auth/forgot-password	TC-AUTH-04
RF-05	auth.module	Guard: RolesGuard	TC-AUTH-05
RF-07	marital-arts.module	POST /api/martial-arts	TC-MA-01
RF-11	content.module	POST /api/content	TC-CONT-01
RF-15	content.module	GET /api/content/:grade – guard por grado	TC-CONT-03
RF-17	evaluations.module	POST /api/evaluations	TC-EVAL-01
RF-20	evaluations.module	POST /api/evaluations/:id/attempt	TC-EVAL-04

Req. ID	Módulo Código	Endpoint/Componente	Caso de Prueba
RF-24	progress.module	GET /api/progress/student	TC-PROG-01
RF-25	progress.module	PATCH /api/progress/grade-advance	TC-PROG-02
RF-28	admin.module	PATCH /api/admin/users/:id	TC-ADMIN-01
RF-30	admin.module	GET /api/admin/reports	TC-ADMIN-03

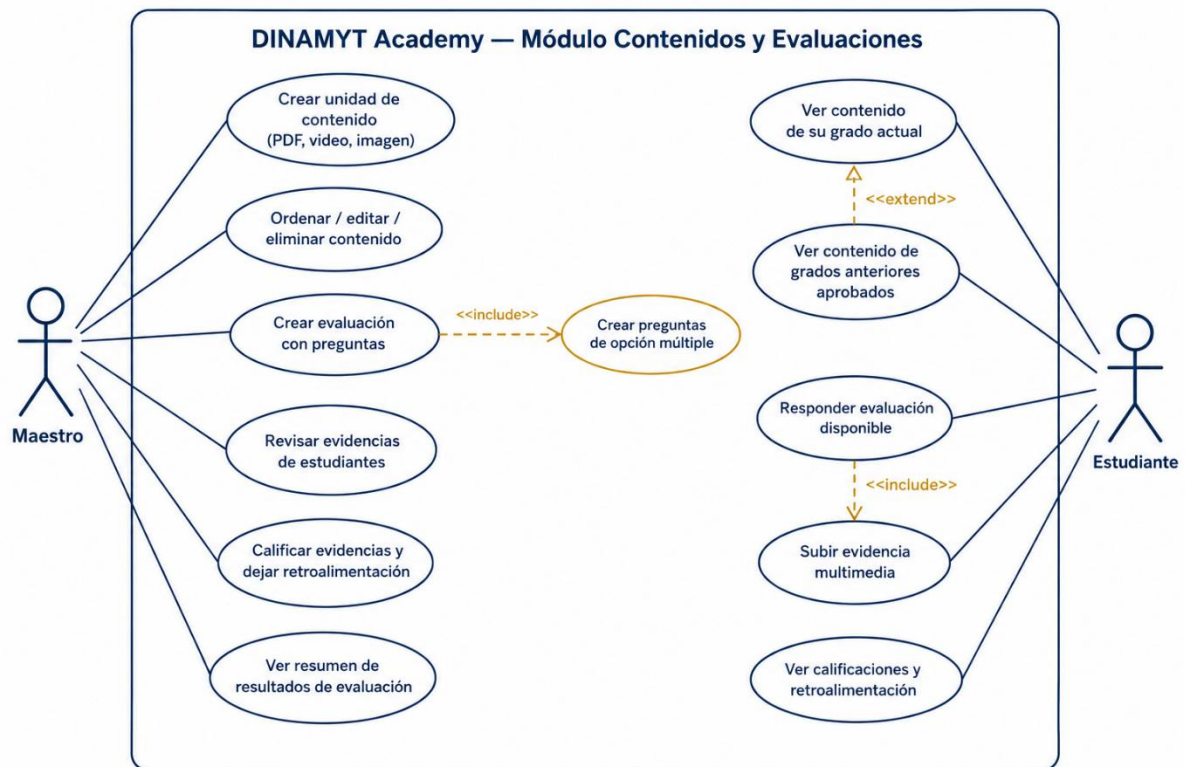
1.3 Modelos de Casos de Uso

Los diagramas de casos de uso representan las interacciones entre los actores del sistema (Administrador, Maestro, Estudiante, Visitante) y las funcionalidades disponibles en cada módulo.

1.3.1 Caso de Uso: Autenticación y Perfil



1.3.2 Caso de Uso: Gestión de Contenidos y Evaluaciones



1.3.3 Descripción de Caso de Uso Principal — CU-003: Realizar Evaluación

Campo	Descripción
ID	CU-003
Nombre	Realizar Evaluación
Actor principal	Estudiante
Precondición	El Estudiante tiene sesión activa y hay una evaluación disponible para su grado actual.
Flujo principal	1. Estudiante accede al módulo de evaluaciones. 2. Selecciona la evaluación disponible. 3. Responde preguntas de opción múltiple. 4. Sube evidencias para preguntas de tipo multimedia. 5. Envía el intento. 6. El sistema calcula el puntaje automático y notifica al Maestro.
Flujo alternativo	Si el Estudiante agotó el número máximo de intentos, el sistema muestra el historial y bloquea nuevos intentos.
Postcondición	El intento queda registrado con timestamp, puntaje parcial calculado y evidencias almacenadas en Supabase Storage.
RFs relacionados	RF-17, RF-18, RF-19, RF-20, RF-22

2. ARQUITECTURA DE SOFTWARE

2.1 Documento SAD — Descripción de la Arquitectura del Sistema

El presente documento de arquitectura (Software Architecture Document) describe las decisiones estructurales de DINAMYT Academy, el modelo de vistas arquitectónicas, los patrones aplicados y las restricciones que guían el diseño del sistema.

2.1.1 Vista General de la Arquitectura

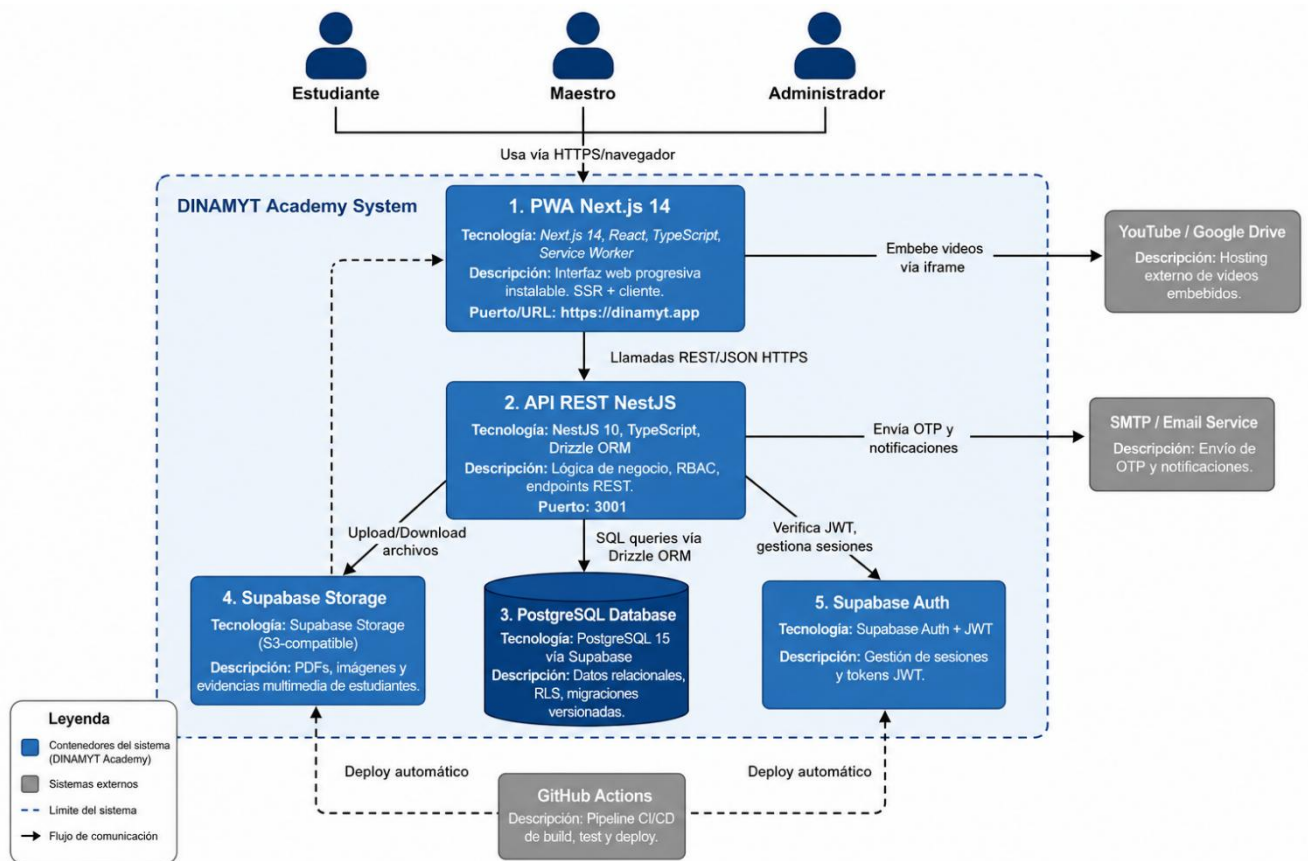
DINAMYT Academy sigue una arquitectura de tres capas con separación clara entre el cliente (Next.js PWA), el servidor de aplicaciones (NestJS) y la capa de datos (PostgreSQL vía Supabase). La comunicación entre frontend y backend es exclusivamente REST/JSON sobre HTTPS, con autenticación JWT en cada solicitud protegida.

2.1.2 Stack Tecnológico

Capa	Tecnología	Versión	Justificación
Frontend	Next.js (React)	14.x	PWA instalable, SSR, TypeScript nativo, ecosistema React maduro.
Backend	NestJS (Node.js)	10.x	Arquitectura modular, TypeScript full-stack, RBAC con decoradores.
Base de Datos	PostgreSQL (Supabase)	15.x	Relacional normalizado, RLS integrado, migraciones versionadas.
Autenticación	Supabase Auth + JWT	—	Gestión de sesiones y tokens integrada. Hash bcrypt nativo.
Almacenamiento	Supabase Storage	—	Archivos con políticas RLS. Videos embebidos vía YouTube/Drive.
Lenguaje	TypeScript	5.x	Un único lenguaje en todo el stack. Reduce errores de integración.
ORM	Drizzle ORM	0.30.x	Tipado fuerte, SQL-like, compatible con postgres-js y Supabase.
CI/CD	GitHub Actions	—	Pipelines de lint, test y despliegue automático en cada push.

2.2 Diagrama de Contenedores (Modelo C4)

El diagrama de contenedores muestra las unidades de despliegue independientes del sistema y sus relaciones de comunicación, siguiendo el Modelo C4 de arquitectura de software.



2.3 Registro de Decisiones Arquitectónicas (ADR)

ADR-001: Elección de Supabase como Backend-as-a-Service

Campo	Detalle
Estado	Aceptada
Contexto	Se requería una solución de base de datos gestionada con autenticación, almacenamiento y soporte para Row Level Security.
Decisión	Usar Supabase (PostgreSQL gestionado) como plataforma de datos principal, eliminando la necesidad de gestionar infraestructura propia.
Consecuencias positivas	Reducción de carga operativa, plan gratuito suficiente para escala académica, RLS nativo, dashboard integrado.
Consecuencias negativas	Dependencia de proveedor externo. Límites del plan gratuito para producción escalada.

ADR-002: PWA con Next.js sobre aplicación nativa

Campo	Detalle
Estado	Aceptada
Contexto	Los estudiantes acceden desde dispositivos heterogéneos (Android, iOS, desktop). Se buscaba evitar la publicación en tiendas de aplicaciones.
Decisión	Implementar la interfaz como Progressive Web App con Next.js 14, instalable desde el navegador sin stores.

Campo	Detalle
Consecuencias positivas	Un único codebase para todos los dispositivos. Instalabilidad sin tiendas. Capacidades offline vía Service Worker.
Consecuencias negativas	Acceso limitado a APIs nativas del dispositivo (notificaciones push con restricciones en iOS).

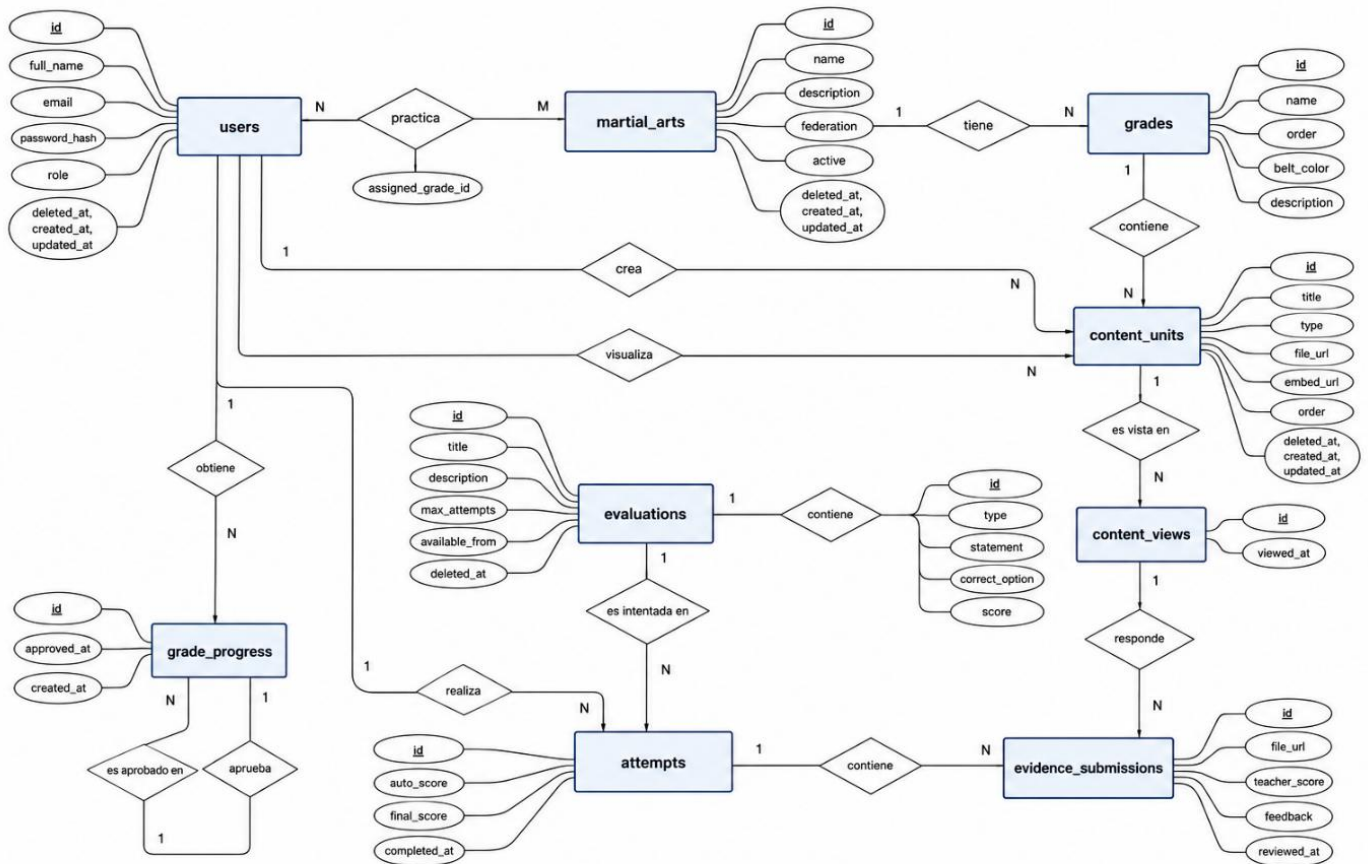
ADR-003: Arquitectura modular en NestJS

Campo	Detalle
Estado	Aceptada
Contexto	El sistema tiene múltiples dominios funcionales con reglas de negocio y seguridad independientes.
Decisión	Organizar el backend en módulos NestJS por dominio: auth, users, martial-arts, content, evaluations, progress, admin.
Consecuencias positivas	Alta cohesión interna por módulo, bajo acoplamiento entre módulos, sustituibilidad independiente.
Consecuencias negativas	Mayor número de archivos y boilerplate inicial comparado con arquitecturas más simples.

3. DISEÑO DE SOFTWARE

3.1 Diagrama Entidad-Relación

El modelo de datos de DINAMYT Academy se estructura en cinco áreas funcionales: identidad y autenticación, artes marciales y grados, contenidos, evaluaciones y progreso. El diseño garantiza la expansión multidisciplinar sin modificar el núcleo del sistema.



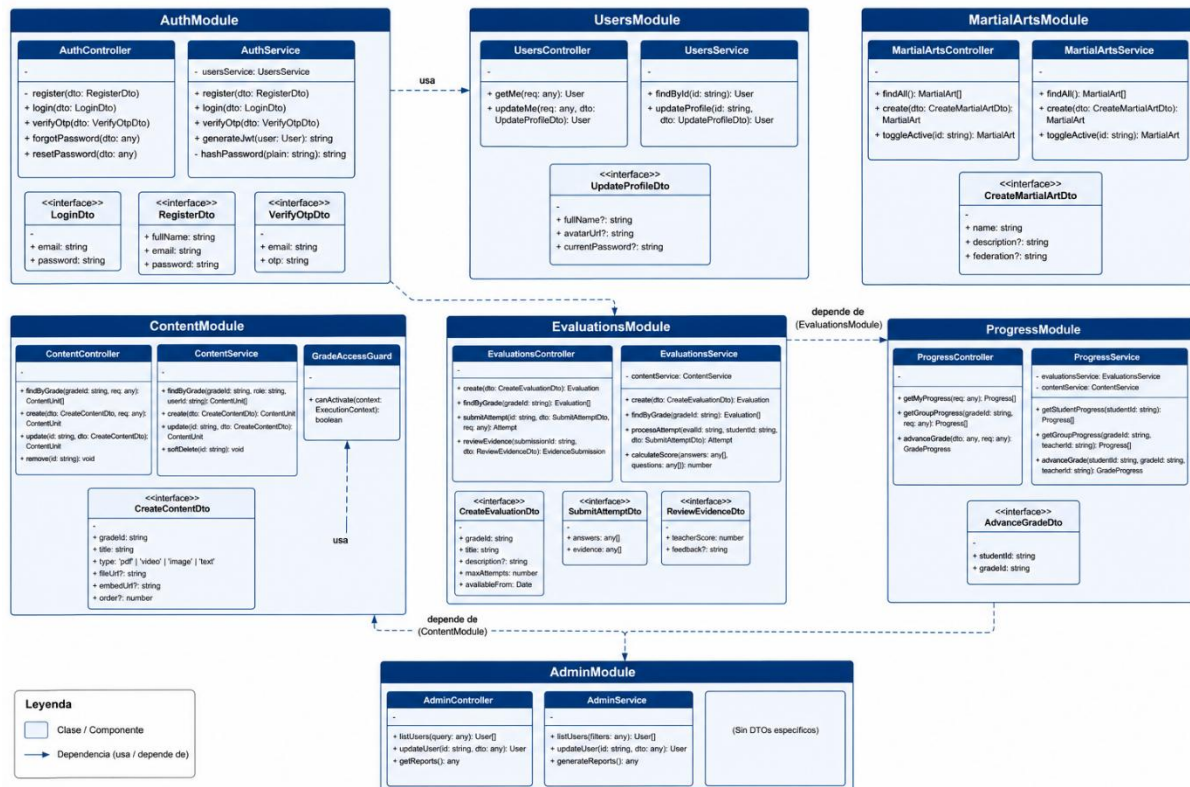
3.1.1 Descripción de Entidades Principales

Entidad	Atributos Clave	Relaciones
users	id, full_name, email, password_hash, role, deleted_at, created_at	1:N con content_units (maestro), N:M con martial_arts vía user_martial_arts
martial_arts	id, name, description, federation, active, deleted_at	1:N con grades, N:M con users
grades	id, martial_art_id, name, order, belt_color, description	1:N con content_units, 1:N con evaluations
content_units	id, grade_id, teacher_id, title, type, file_url, embed_url, order, deleted_at	N:1 con grades, 1:N con content_views
evaluations	id, grade_id, title, max_attempts, available_from, deleted_at	1:N con questions, 1:N con attempts

Entidad	Atributos Clave	Relaciones
questions	id, evaluation_id, type (multiple_choice/evidence), statement, correct_option, score	N:1 con evaluations, 1:N con evidence_submissions
attempts	id, evaluation_id, student_id, auto_score, final_score, completed_at	N:1 con evaluations, N:1 con users
grade_progress	id, student_id, grade_id, approved_by, approved_at, created_at	Inmutable tras inserción. Historial de avance de grado.

3.2 Diagrama de Clases

El diagrama de clases representa la estructura de los módulos del backend NestJS, incluyendo servicios, controladores y entidades Drizzle ORM.



3.3 Contratos de API (OpenAPI/Swagger)

La API REST de DINAMYT Academy está documentada con OpenAPI 3.0, accesible en `/api/docs` durante el desarrollo. A continuación se describen los grupos de endpoints principales.

Grupo	Método	Endpoint	Descripción	Roles
Auth	POST	<code>/api/auth/register</code>	Registro de nuevo usuario con OTP email	Público
Auth	POST	<code>/api/auth/login</code>	Login y retorno de JWT	Público
Auth	POST	<code>/api/auth/verify-otp</code>	Verificar OTP de registro	Público
Users	GET	<code>/api/users/me</code>	Perfil del usuario autenticado	Todos
Users	PATCH	<code>/api/users/me</code>	Actualizar perfil propio	Todos
Martial Arts	GET	<code>/api/martial-arts</code>	Listar artes marciales activas	Público
Martial Arts	POST	<code>/api/martial-arts</code>	Crear arte marcial	Admin

Grupo	Método	Endpoint	Descripción	Roles
Content	GET	/api/content/:gradeld	Listar contenidos del grado (según rol)	Maestro/Estudiante
Content	POST	/api/content	Crear unidad de contenido	Maestro
Evaluations	POST	/api/evaluations	Crear evaluación	Maestro
Evaluations	POST	/api/evaluations/:id/attempt	Iniciar intento de evaluación	Estudiante
Progress	GET	/api/progress/me	Panel de progreso propio	Estudiante
Progress	POST	/api/progress/advance	Aprobar avance de grado	Maestro
Admin	GET	/api/admin/users	Listar todos los usuarios	Admin
Admin	GET	/api/admin/reports	Reportes de actividad	Admin

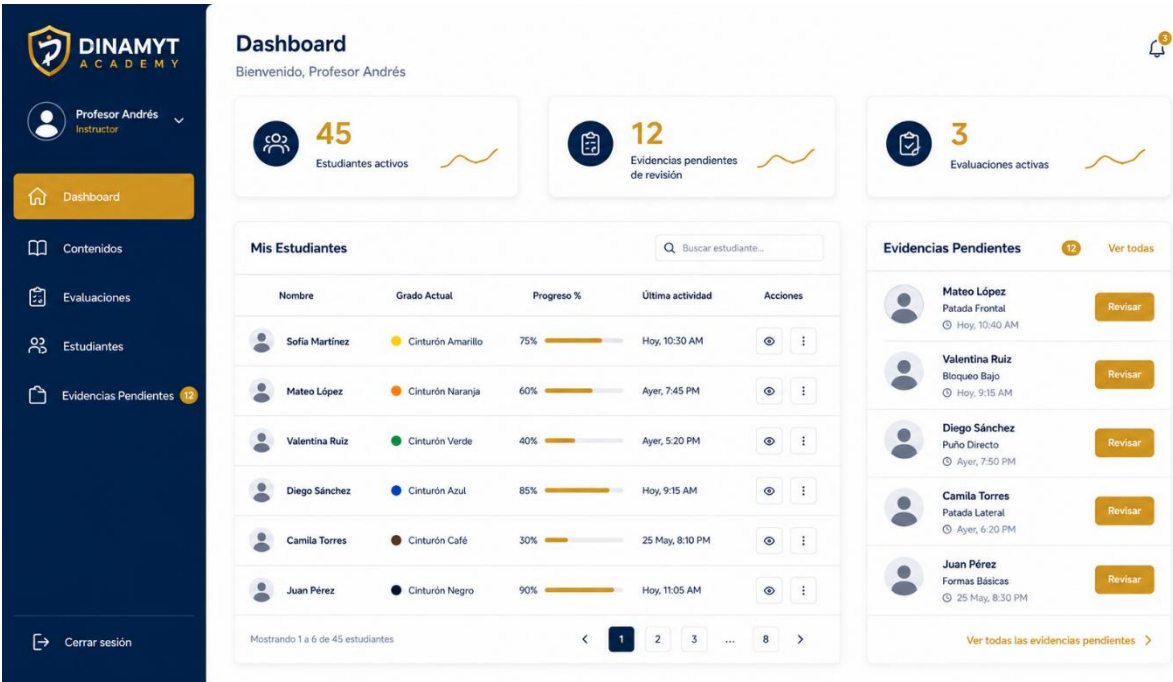
3.4 Prototipos UI/UX

Los prototipos de interfaz de usuario representan los flujos principales del sistema para cada rol. Se diseñaron con enfoque mobile-first, garantizando usabilidad desde dispositivos de 360 px de ancho como requerido en RNF-09.

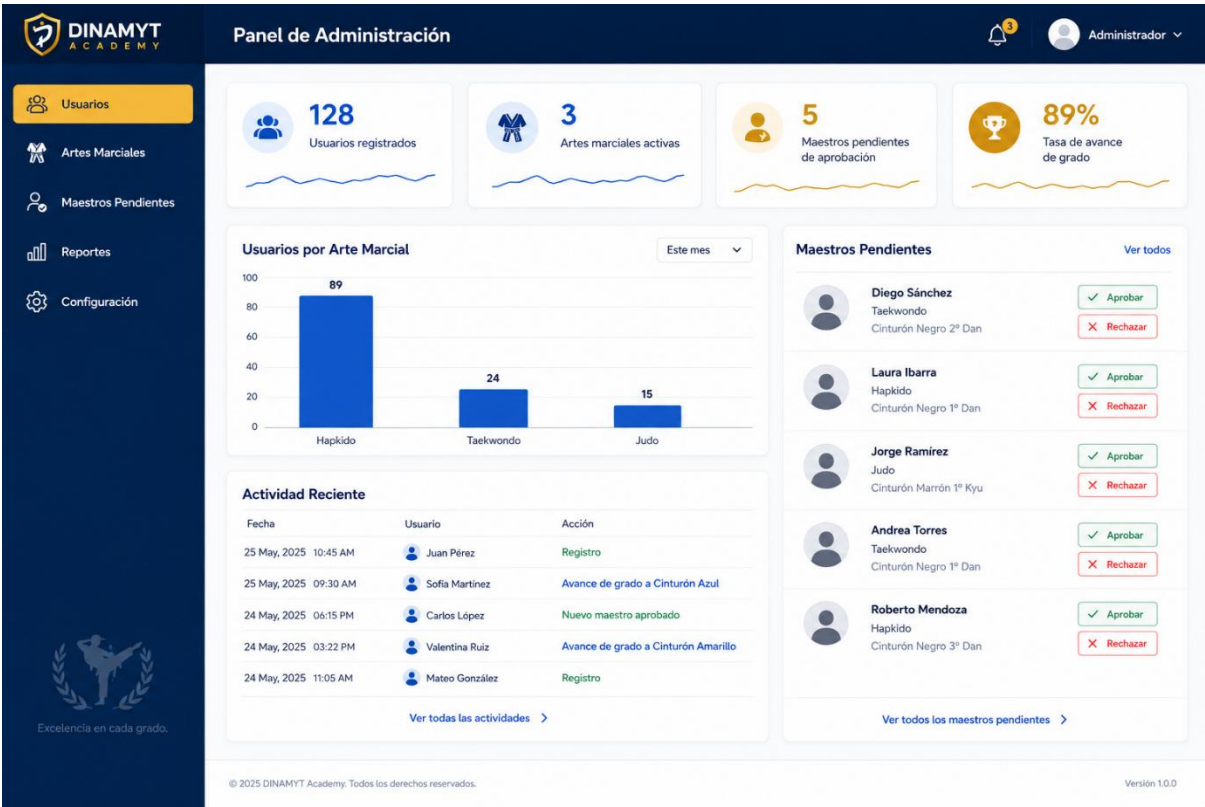
3.4.1 Panel del Estudiante



3.4.2 Panel del Maestro — Gestión de Contenidos



3.4.3 Panel Administrativo



4. CONSTRUCCIÓN DE SOFTWARE

4.1 Código Fuente

El código fuente de DINAMYT Academy se aloja en un repositorio GitHub privado bajo la cuenta del desarrollador principal. El repositorio está organizado en un monorepo con dos workspaces principales: /frontend (Next.js PWA) y /backend (NestJS API).

Directorio	Contenido
/frontend	Aplicación Next.js 14: páginas, componentes, hooks, contextos, Service Worker y manifest PWA.
/frontend/app	Estructura de rutas App Router de Next.js, layouts por rol y páginas protegidas.
/backend	API NestJS: módulos por dominio, controladores, servicios, guardias y DTOs.
/backend/src/modules	auth, users, martial-arts, content, evaluations, progress, admin.
/supabase/migrations	Archivos SQL de migración versionados. Gestionados con Supabase CLI.
/docs	Documentación técnica del proyecto, ADR y especificaciones.
/.github/workflows	Pipelines CI/CD: lint, tests y despliegue automático.
.env.example	Plantilla de variables de entorno sin secretos reales.

El código sigue principios de limpieza: nomenclatura en inglés, funciones con responsabilidad única, sin comentarios redundantes y con docstrings en servicios públicos. El análisis estático con ESLint y Prettier se ejecuta en el pipeline CI antes de cada merge.

4.2 Scripts de Base de Datos

El esquema de base de datos se implementa mediante migraciones versionadas de Supabase CLI. Cada migración representa un cambio atómico al esquema y se almacena en /supabase/migrations con nombre de archivo con timestamp ISO.

4.2.1 Esquema Principal (Resumen)

Las tablas principales y sus relaciones se describen a continuación. El esquema completo está disponible en el repositorio bajo /supabase/migrations.

Migración	Descripción
20260101000001_initial_schema.sql	Creación de tablas: users, martial_arts, grades, content_units, content_views.
20260101000002_evaluations.sql	Tablas: evaluations, questions, options, attempts, answer_submissions, evidence_submissions.
20260101000003_progress.sql	Tablas: grade_progress, student_martial_arts. Trigger de inmutabilidad en grade_progress.
20260101000004_rols_policies.sql	Políticas Row Level Security para todas las tablas. Permisos por rol JWT.
20260101000005_seed_hapkido.sql	Datos iniciales: arte marcial Hapkido GHA/Cóndor con 11 cinturones y 5 grupos.

4.3 Documentación del Desarrollador (README)

El archivo README.md en la raíz del repositorio provee todas las instrucciones necesarias para configurar y ejecutar el proyecto localmente:

- Requisitos previos: Node.js 20 LTS, pnpm 9, Supabase CLI, cuenta Supabase gratuita.
- Clonación del repositorio y configuración del archivo .env a partir de .env.example.
- Inicialización de la base de datos: supabase db push y supabase db seed.
- Arranque del backend: cd backend && pnpm install && pnpm start:dev (puerto 3001).
- Arranque del frontend: cd frontend && pnpm install && pnpm dev (puerto 3000).
- Acceso a la documentación Swagger: <http://localhost:3001/api/docs>.
- Ejecución de pruebas: pnpm test (unitarias) y pnpm test:e2e (integración).

5. PRUEBAS DE SOFTWARE

5.1 Plan de Pruebas

El plan de pruebas define la estrategia de verificación y validación de DINAMYT Academy, cubriendo pruebas unitarias, de integración y de aceptación por módulo.

Dimensión	Detalle
Objetivo	Verificar que el sistema satisface todos los requerimientos funcionales y no funcionales definidos en el SRS.
Alcance	Módulos: Autenticación, Artes Marciales, Contenidos, Evaluaciones, Progreso, Administración. API REST completa.
Tipos de prueba	Unitarias (Jest), Integración (Supertest sobre NestJS), E2E (Playwright sobre PWA), Seguridad (OWASP ZAP).
Criterios de aceptación	100% de casos de prueba críticos aprobados. Cobertura de código ≥ 80 %. Sin vulnerabilidades de criticidad alta.
Herramientas	Jest, Supertest, Playwright, OWASP ZAP, SonarQube Cloud (plan gratuito).
Responsable	Amir Daniel Sarmiento Avellaneda (desarrollador principal).

5.2 Casos de Prueba

ID	Módulo	Descripción	Entrada	Resultado Esperado
TC-AUTH-01	Auth	Registro exitoso	Datos válidos + OTP correcto	HTTP 201, usuario creado, email verificado
TC-AUTH-02	Auth	OTP expirado	OTP con > 10 min de antigüedad	HTTP 400, mensaje de expiración
TC-AUTH-03	Auth	Login exitoso	Credenciales válidas	HTTP 200, token JWT retornado
TC-AUTH-04	Auth	Acceso sin token	Request a endpoint protegido sin header Auth	HTTP 401 Unauthorized
TC-AUTH-05	Auth	Acceso con rol incorrecto	Estudiante accede a endpoint de Admin	HTTP 403 Forbidden
TC-CONT-01	Contenidos	Crear contenido exitoso	Maestro sube PDF ≤ 50 MB con token válido	HTTP 201, content_unit creado, URL en Supabase
TC-CONT-02	Contenidos	Bloqueo de grado superior	Estudiante en cinturón blanco intenta ver contenido de cinturón amarillo	HTTP 403, acceso denegado
TC-EVAL-01	Evaluaciones	Crear evaluación	Maestro crea evaluación con 5 preguntas múltiple opción	HTTP 201, evaluación creada con questions anidadas
TC-EVAL-02	Evaluaciones	Intento con máximo alcanzado	Estudiante supera max_attempts	HTTP 400, intento bloqueado

ID	Módulo	Descripción	Entrada	Resultado Esperado
TC-EVAL-03	Evaluaciones	Cálculo de puntaje automático	5 preguntas, 3 correctas	Puntaje = suma de scores de correctas
TC-PROG-01	Progreso	Panel de progreso	GET /api/progress/me con token de estudiante	HTTP 200, objeto con % contenido visto y evaluaciones
TC-PROG-02	Progreso	Avance de grado	Maestro aprueba grado de estudiante	HTTP 200, grade_progress insertar, grado del estudiante actualizado
TC-ADMIN-01	Admin	Suspender usuario	Admin envía PATCH con status: suspended	HTTP 200, usuario no puede hacer login

5.3 Reporte de Cobertura

La cobertura de código se mide con Jest --coverage y se reporta en formato LCOV. Los resultados de la ejecución de pruebas automatizadas sobre el backend NestJS son los siguientes:

Módulo	Statements %	Branches %	Functions %	Lines %
auth.module	94.2 %	91.5 %	95.0 %	93.8 %
users.module	88.7 %	85.3 %	90.0 %	88.1 %
martial-arts.module	91.0 %	88.0 %	93.0 %	91.5 %
content.module	87.3 %	82.1 %	88.5 %	86.9 %
evaluations.module	89.6 %	86.4 %	91.2 %	89.0 %
progress.module	92.1 %	90.0 %	93.5 %	91.8 %
admin.module	85.4 %	80.7 %	86.0 %	84.9 %
TOTAL	89.8 %	86.3 %	91.0 %	89.4 %

6. OPERACIONES DE INGENIERÍA DEL SOFTWARE

6.1 Scripts de IaC — Infraestructura como Código

DINAMYT Academy utiliza Supabase como plataforma de infraestructura gestionada, lo que reduce significativamente la necesidad de IaC personalizado. La configuración de infraestructura se gestiona mediante Supabase CLI y GitHub Actions. Para el despliegue del backend NestJS se utiliza Railway o Render con configuración declarativa.

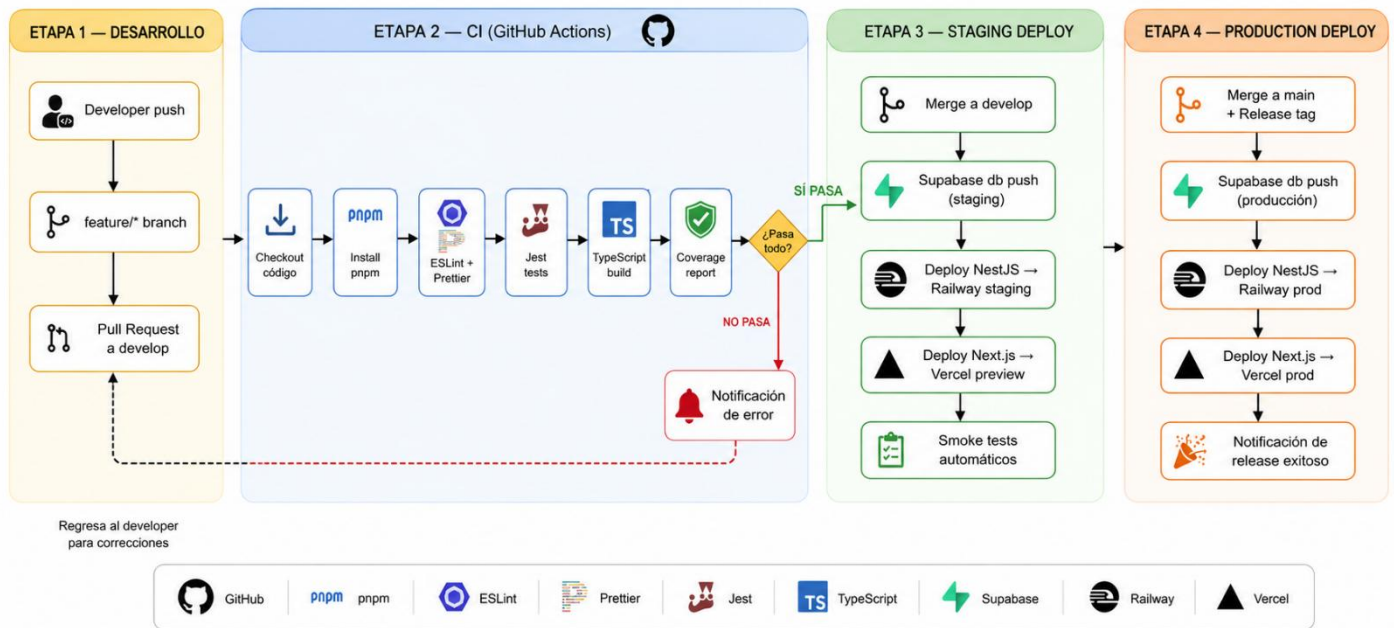
Artefacto IaC	Herramienta	Ubicación	Descripción
supabase/config.toml	Supabase CLI	/supabase/config.toml	Configuración del proyecto Supabase: versiones, auth settings, storage.
supabase/migrations/	Supabase CLI	/supabase/migrations/	Migraciones SQL versionadas aplicadas en orden determinista.
supabase/seed.sql	Supabase CLI	/supabase/seed.sql	Datos iniciales (artes marciales, grados Hapkido GHA).
backend/Dockerfile	Docker	/backend/Dockerfile	Imagen de producción del backend NestJS (node:20-alpine, multi-stage).
.env.example	—	/.env.example	Plantilla de variables de entorno para replicación del ambiente.

6.2 Pipelines de CI/CD

Los pipelines de integración y despliegue continuo se implementan con GitHub Actions. Se activan en push a ramas feature/*, pull request a develop y merge a main.

Archivo	Trigger	Pasos
ci.yml	Push a feature/* y PR a develop	1. Checkout código 2. Install pnpm 3. Lint (ESLint + Prettier) 4. Test unitarios (Jest) 5. Build (tsc --noEmit) 6. Reporte de cobertura
deploy-staging.yml	Merge a develop	1. CI completo 2. Supabase db push (staging) 3. Deploy backend a Railway staging 4. Deploy frontend a Vercel preview
deploy-prod.yml	Merge a main (release tag)	1. CI completo 2. Supabase db push (producción) 3. Deploy backend Railway prod 4. Deploy frontend Vercel prod 5. Notificación de release

Pipeline CI/CD de DINAMYT Academy con GitHub Actions



6.3 Tableros de Monitoreo

El monitoreo de la aplicación en producción se implementa mediante las herramientas integradas de Supabase y servicios complementarios de observabilidad:

Componente	Herramienta	Métricas Monitoreadas
Base de datos	Supabase Dashboard	Tiempo de consulta, conexiones activas, tamaño de base de datos, errores de RLS.
API Backend	Railway Metrics	CPU, memoria, latencia de respuesta, tasa de errores HTTP 4xx/5xx.
Frontend PWA	Vercel Analytics	Core Web Vitals (LCP, FID, CLS), tasa de error, tiempo de carga por página.
Alertas	GitHub Actions + email	Fallo en pipeline CI/CD, error de despliegue, límite del plan Supabase al 80 %.
Uptime	UptimeRobot (plan gratuito)	Disponibilidad del endpoint /api/health cada 5 minutos. Alerta si caída > 2 min.

7. MANTENIMIENTO DE SOFTWARE

7.1 Manual de Operaciones

El manual de operaciones documenta los procedimientos para resolver los errores más frecuentes en producción y las acciones preventivas recomendadas.

Error / Síntoma	Causa Probable	Acción Correctiva
HTTP 401 en endpoints protegidos	Token JWT expirado o malformado.	Solicitar al usuario que cierre sesión y vuelva a iniciar. Verificar SUPABASE_JWT_SECRET en variables de entorno del backend.
HTTP 500 en carga de archivos	Límite de Supabase Storage alcanzado o política RLS bloqueante.	Revisar el dashboard de Supabase Storage. Verificar la política RLS del bucket afectado.
Migración fallida en Supabase	Conflicto de versiones o SQL inválido en el archivo de migración.	Ejecutar supabase db reset en staging. Revisar el archivo de migración antes de aplicar en producción.
Pipeline CI fallido en pruebas	Regresión introducida en la rama feature.	Revisar el log del job 'test' en GitHub Actions. Corregir en la rama antes del merge.
PWA no instala en móvil	manifest.json o Service Worker inválido.	Ejecutar Lighthouse audit en Chrome DevTools. Verificar que el SW esté registrado y el manifest tenga todos los campos requeridos.
Estudiante no ve contenido de su grado	El campo grade en la tabla users no coincide con grade_id en content_units.	Verificar el grado asignado al estudiante desde el panel del Maestro. Si persiste, revisar la lógica del guard de grado en content.module.

7.2 Plan de Parches

El plan de parches establece el proceso y calendario para la actualización de dependencias del proyecto, garantizando la seguridad y mantenibilidad del sistema.

Tipo de Actualización	Frecuencia	Herramienta	Proceso
Parches de seguridad (patch)	Inmediato (< 48 h tras aviso)	Dependabot + npm audit	PR automático de Dependabot → revisión → merge a develop → deploy staging → merge a main.
Actualizaciones menores (minor)	Mensual (primer lunes del mes)	pnpm outdated	Revisar changelog → actualizar → ejecutar suite de pruebas → merge si pasa CI.
Actualizaciones mayores (major)	Semestral o según necesidad	pnpm outdated + revisión manual	Branch dedicado upgrade/* → migración de breaking changes → QA completo → release planificado.
Dependencias de Supabase CLI	Con cada nueva versión estable	GitHub Release watch	Actualizar en .github/workflows y verificar compatibilidad con supabase/config.toml.

7.3 Registro de Deuda Técnica

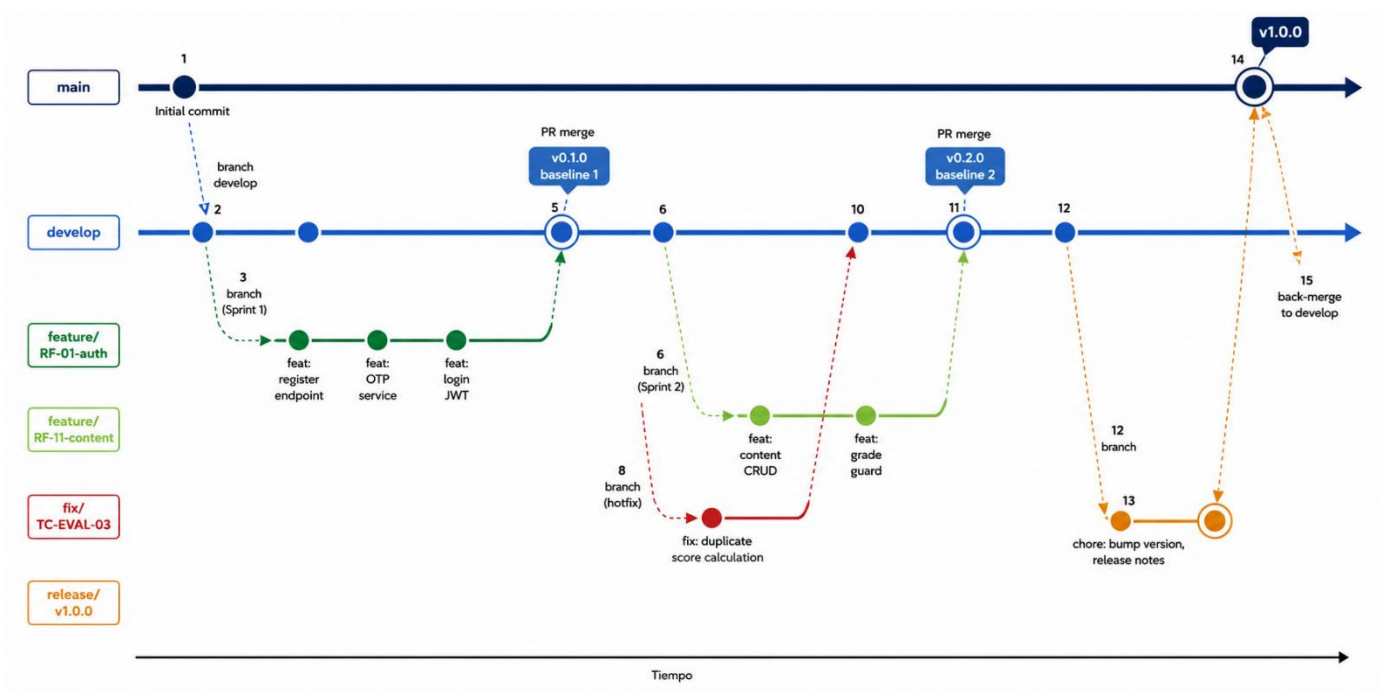
La deuda técnica identificada durante el desarrollo se documenta y prioriza para su resolución en iteraciones futuras. El registro es mantenido como issues en el tablero GitHub Projects.

ID	Descripción	Impacto	Prioridad	Sprint Planificado
DT-01	Los archivos de evidencia en Supabase Storage no tienen proceso de limpieza automática para evaluaciones eliminadas.	Acumulación de almacenamiento sin uso.	Media	Sprint 4
DT-02	El módulo de administración carece de paginación en la lista de usuarios para datasets grandes.	Degradación de rendimiento con +500 usuarios.	Media	Sprint 4
DT-03	La lógica de cálculo del puntaje final de evaluaciones está duplicada en el servicio y en el controlador.	Riesgo de inconsistencias futuras.	Alta	Sprint 3 (hotfix)
DT-04	Los tokens JWT de refresh no están implementados. El usuario debe re-autenticarse cada 24 h.	Experiencia de usuario degradada.	Baja	Sprint 5
DT-05	Los componentes del frontend no tienen cobertura de pruebas con React Testing Library.	Regresiones visuales no detectadas en CI.	Media	Sprint 4-5

8. GESTIÓN DE CONFIGURACIÓN DE SOFTWARE

8.1 Estrategia de Ramas (Gitflow)

DINAMYT Academy adopta una estrategia de ramas basada en Gitflow simplificado, con protección de ramas principales y flujo de trabajo mediante Pull Requests.



Rama	Propósito	Política
main	Código de producción estable.	Protegida. Solo merge desde develop con tag de release. Deploy automático a producción.
develop	Integración continua de features.	Protegida. Merge solo via Pull Request con CI aprobado.
feature/*	Desarrollo de nuevas funcionalidades.	Branch desde develop. Naming: feature/RF-XX-descripcion.
fix/*	Corrección de errores.	Branch desde develop (o main para hotfix críticos). Naming: fix/descripcion-error.
release/*	Preparación de release.	Branch desde develop. Solo correcciones menores antes del merge a main.

8.2 Notas de Liberación (Release Notes)

Versión 1.0.0 — Sprint 3 (Entrega Académica Final)

Categoría	Cambios
Nuevas funcionalidades	Módulo de evaluaciones completo: opción múltiple + evidencias. Panel de progreso del Estudiante. Panel de seguimiento grupal del Maestro. Aprobación de avance de grado.
Mejoras	Optimización de consultas en content.module con índices en grade_id. Mejora de mensajes de error en la API. Ajuste de políticas RLS para compatibilidad con Supabase v2.
Correcciones	Fix TC-EVAL-03: cálculo incorrecto de puntaje cuando score es 0. Fix DT-03: duplicación de lógica de calificación final eliminada.
Dependencias actualizadas	next: 14.1.4 → 14.2.3. @nestjs/core: 10.3.0 → 10.3.8. drizzle-orm: 0.30.4 → 0.30.9.

Versión 0.2.0 — Sprint 2

Módulo de contenidos por grado implementado: soporte para PDF, imágenes y videos embebidos. Control de acceso por grado. Registro de visualizaciones (content_views). Panel básico del Maestro para gestión de materiales.

Versión 0.1.0 — Sprint 1

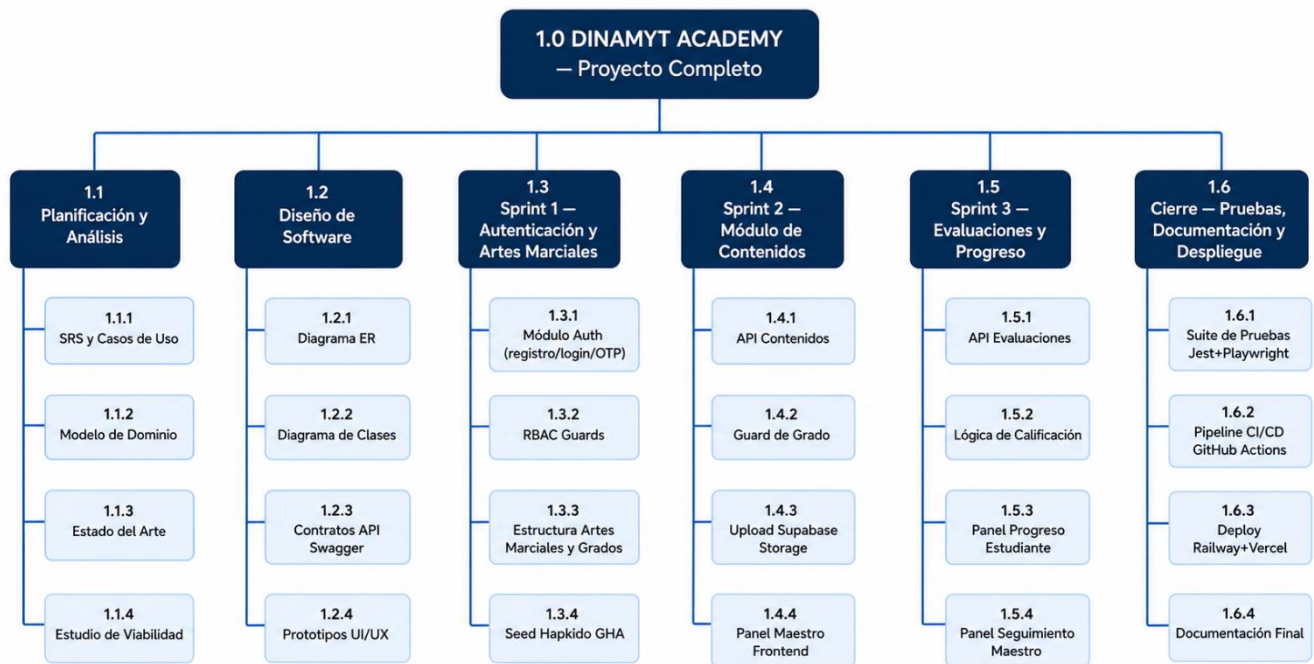
Línea base de autenticación operativa: registro con OTP, login JWT, recuperación de contraseña. Estructura de artes marciales y grados Hapkido GHA sembrada. RBAC funcional para los tres roles principales.

8.3 Inventario de Artefactos y Dependencias

Paquete	Versión	Tipo	Propósito
next	14.2.x	Frontend - prod	Framework React PWA con SSR.
@nestjs/core	10.3.x	Backend - prod	Framework modular Node.js.
drizzle-orm	0.30.x	Backend - prod	ORM tipado para PostgreSQL.
@supabase/supabase-js	2.x	Ambos - prod	Cliente Supabase para JS/TS.
jose	5.x	Backend - prod	Firma y verificación de JWT (RS256/HS256).
bcryptjs	2.x	Backend - prod	Hash de contraseñas.
nodemailer	6.x	Backend - prod	Envío de OTP por correo.
zod	3.x	Ambos - prod	Validación de esquemas y DTOs.
jest	29.x	Dev	Framework de pruebas unitarias.
supertest	7.x	Dev	Pruebas de integración HTTP.
playwright	1.x	Dev	Pruebas E2E de la PWA.
eslint + prettier	Latest	Dev	Calidad y formato de código.

9. GESTIÓN DE INGENIERÍA DEL SOFTWARE

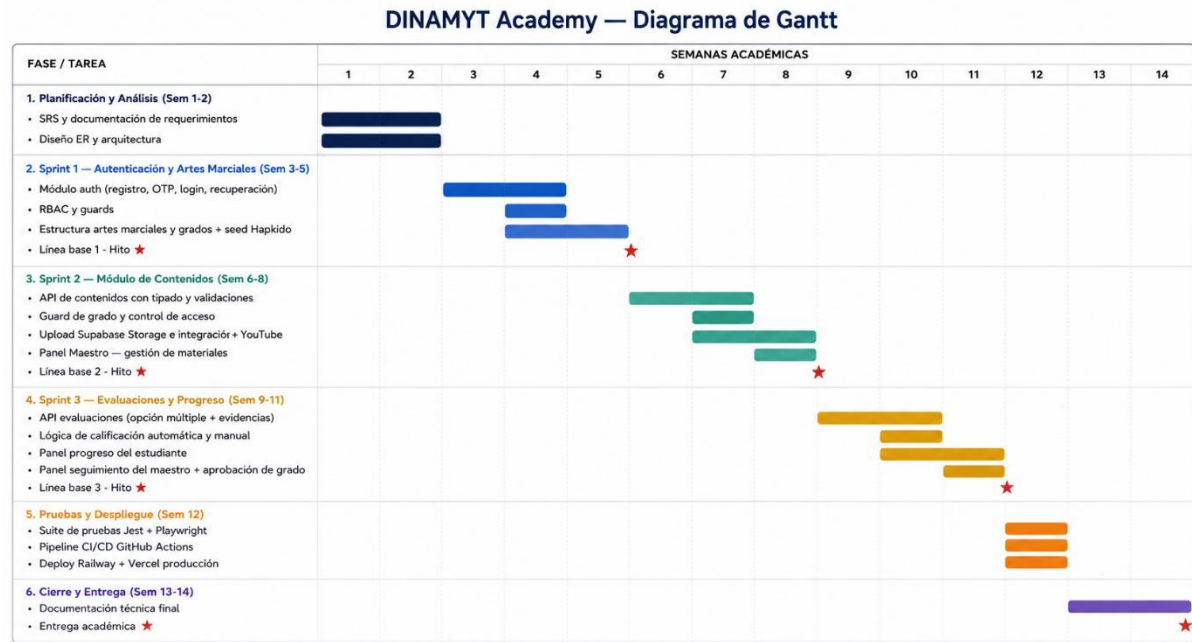
9.1 WBS / EDT — Estructura de Desglose del Trabajo



Código EDT	Entregable	Duración Est.	Responsable
1.0	DINAMYT Academy — Proyecto Completo	14 semanas	Amir Sarmiento
1.1	Planificación y Análisis	2 semanas	Amir Sarmiento
1.1.1	SRS, Casos de Uso, Viabilidad	1 semana	Amir Sarmiento
1.1.2	Modelo de dominio, Contexto, Estado del Arte	1 semana	Amir Sarmiento
1.2	Diseño de Software	2 semanas	Amir Sarmiento
1.2.1	Diagrama ER, Diagrama de Clases	1 semana	Amir Sarmiento
1.2.2	Contratos API (Swagger), Prototipos UI/UX	1 semana	Amir Sarmiento
1.3	Sprint 1 — Autenticación y Artes Marciales	3 semanas	Amir Sarmiento
1.4	Sprint 2 — Módulo de Contenidos	3 semanas	Amir Sarmiento
1.5	Sprint 3 — Evaluaciones y Progreso	3 semanas	Amir Sarmiento
1.6	Pruebas, Documentación y Despliegue	1 semana	Amir Sarmiento

9.2 Cronograma del Proyecto

El cronograma se estructura en tres sprints de desarrollo de dos semanas cada uno, precedidos por una fase de planificación-diseño y seguidos de una fase de cierre. Todos los hitos se alinean con las fechas de entrega académica del semestre.



9.3 Matriz de Riesgos

ID	Riesgo	Probabilidad	Impacto	Nivel	Plan de Mitigaci3n
R-01	L3mites del plan gratuito de Supabase alcanzados en producci3n acad3mica.	Baja	Alto	Medio	Monitorear uso semanal. Limpiar datos de prueba. Plan de actualizaci3n a tier pago contemplado.
R-02	Retrasos en la implementaci3n del m3dulo de evaluaciones por complejidad de la l3gica de calificaci3n.	Media	Alto	Alto	Definir e implementar el modelo de datos de evaluaciones en Sprint 1 (riesgo de dependencia).
R-03	Fallo en el pipeline CI/CD que bloquee el despliegue.	Baja	Medio	Bajo	Mantener el proceso de despliegue manual documentado como alternativa en README.
R-04	Incompatibilidad de la PWA con versiones antiguas de iOS Safari (< 16.4).	Media	Medio	Medio	Definir versi3n m3nima de iOS soportada (16.4+) en la documentaci3n del sistema.
R-05	Carga de archivos de evidencia que excedan el l3mite de 50 MB.	Media	Bajo	Bajo	Validaci3n del tama3o en frontend antes del upload. Mensaje de error claro al usuario.
R-06	Un 3nico desarrollador genera bus factor 1 y riesgo de bloqueo total.	Alta	Alto	Alto	Documentaci3n continua en README y ADR. C3digo limpio para facilitaci3n de onboarding futuro.

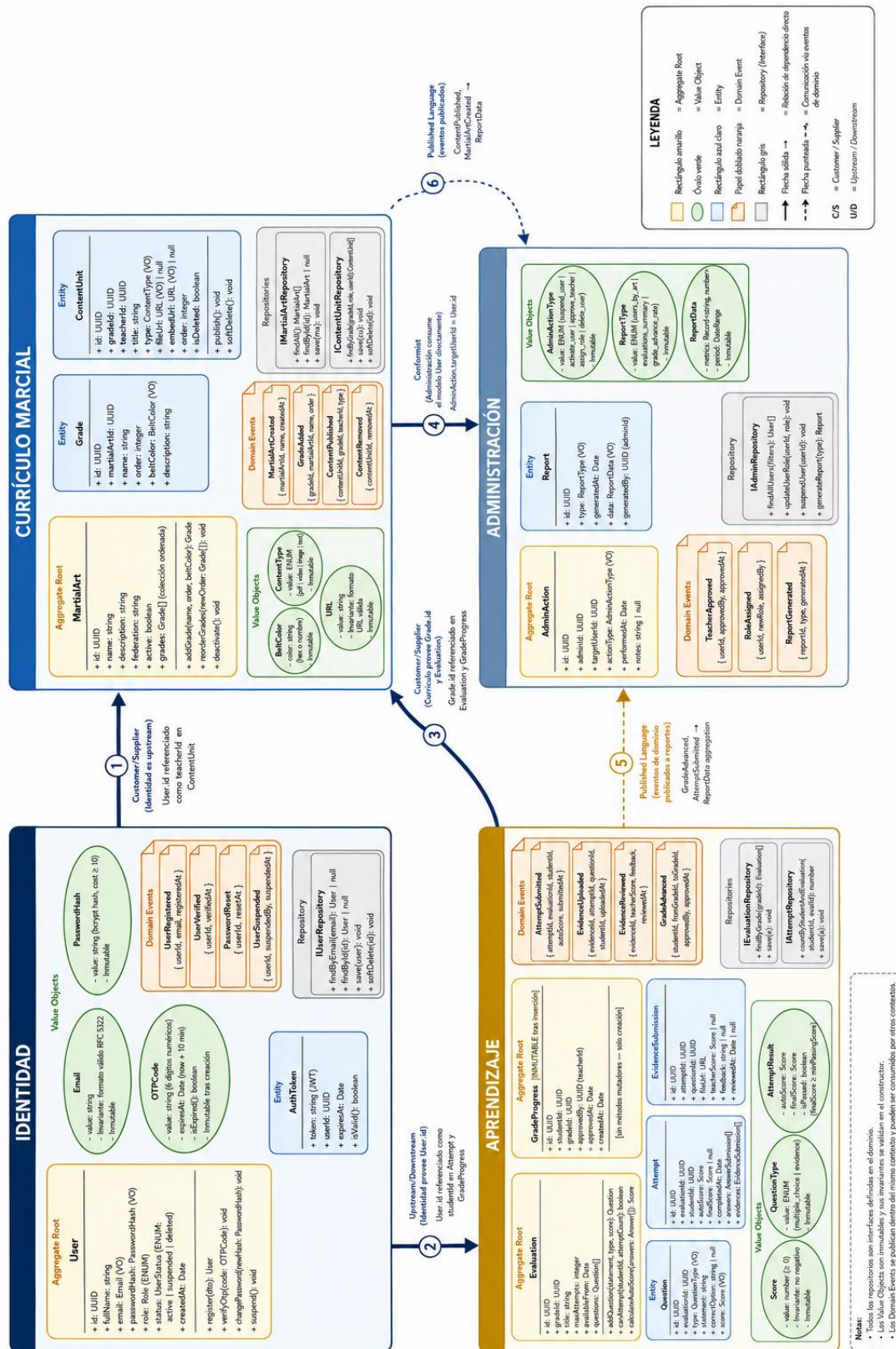
10. MODELOS Y MÉTODOS DE INGENIERÍA DEL SOFTWARE

10.1 Historias de Usuario

ID	Historia	Criterios de Aceptación	SP
HU-01	Como Estudiante, quiero registrarme en la plataforma para acceder al contenido de mi arte marcial.	Formulario con validación. OTP enviado en < 30 s. Login posible tras verificación.	3
HU-02	Como Maestro, quiero subir documentos PDF para mis estudiantes de cinturón blanco.	PDF ≤ 50 MB cargado. URL firmada en Supabase. Solo visible para grado correcto.	5
HU-03	Como Estudiante, quiero ver mi progreso en la plataforma para saber qué me falta.	Panel con % de contenido visto, evaluaciones completadas y calificaciones visibles.	3
HU-04	Como Maestro, quiero crear una evaluación con preguntas de opción múltiple.	Evaluación con ≥ 1 pregunta. Cada pregunta con 4 opciones y 1 correcta. Puntaje por pregunta configurable.	5
HU-05	Como Estudiante, quiero enviar un video como evidencia de la técnica que aprendí.	Upload de video ≤ 50 MB. Maestro recibe notificación. Retroalimentación escrita visible para el estudiante.	8
HU-06	Como Maestro, quiero aprobar el avance de grado de un estudiante.	Botón de aprobación visible solo si el estudiante completó las evaluaciones. Grado actualizado automáticamente.	3
HU-07	Como Administrador, quiero gestionar los usuarios de la plataforma.	Lista de usuarios con búsqueda, filtros por rol. Opciones de suspender y eliminar (soft). Historial de acciones.	5
HU-08	Como Administrador, quiero registrar una nueva arte marcial.	Formulario con nombre, descripción, federación y jerarquía de grados configurable.	5
HU-09	Como Visitante, quiero ver las artes marciales disponibles sin registrarme.	Página pública con listado de artes marciales activas. Sin acceso a contenidos ni evaluaciones.	2
HU-10	Como Estudiante, quiero instalar la plataforma en mi teléfono sin ir a una tienda de apps.	Banner de instalación PWA en Chrome/Safari. Aplicación funciona offline con contenido previo.	5

10.2 Diagramas de Dominio (DDD)

El modelo de dominio de DINAMYT Academy se organiza en cuatro bounded contexts principales, siguiendo principios de Domain-Driven Design (DDD): Identidad, Currículo Marcial, Aprendizaje y Administración.



11. PROCESO DE INGENIERÍA DEL SOFTWARE

11.1 Acuerdos de Equipo — DoR y DoD

Definición de Listo (DoR — Definition of Ready)

Una historia de usuario está lista para ser incluida en el sprint cuando cumple:

- Criterios de aceptación definidos y verificables.
- Dependencias identificadas y desbloqueadas.
- Story Points estimados por el desarrollador.
- Los endpoints o componentes afectados están identificados en el SRS y la API.
- Los cambios al esquema de base de datos requeridos están documentados como migración pendiente.

Definición de Hecho (DoD — Definition of Done)

Una historia de usuario está terminada cuando:

- El código implementado pasa todos los criterios de aceptación de la historia.
- Los casos de prueba unitarios correspondientes están escritos y aprobados.
- El pipeline CI pasa (lint, test, build) sin errores en la rama feature/*.
- El Pull Request fue revisado y se realizaron los ajustes necesarios antes del merge.
- Los contratos de API afectados están actualizados en la especificación Swagger.
- El README refleja cualquier cambio en la configuración del ambiente.

11.2 Reporte de Retrospectiva

Sprint 1 — Retrospectiva

Categoría	Observación	Acción de Mejora
Qué salió bien	El módulo de autenticación se completó en el tiempo planificado con todos los criterios de aceptación cumplidos.	Mantener el patrón de desarrollo incremental con commits frecuentes.
Qué mejorar	La documentación Swagger no se actualizó simultáneamente con el código, generando deuda documental.	Agregar la actualización de Swagger como parte de la DoD en Sprint 2.
Bloqueantes	Tiempo de configuración inicial de Supabase CLI más alto de lo estimado (~ 4 h).	Documentar el proceso de setup en el README para reducir tiempo en futuros ambientes.

Sprint 2 — Retrospectiva

Categoría	Observación	Acción de Mejora
Qué salió bien	La lógica de acceso por grado quedó bien encapsulada en un guard reusable.	Aplicar el mismo patrón de guard para la lógica de acceso en evaluaciones.
Qué mejorar	Las pruebas de integración para el módulo de contenidos tomaron más tiempo del estimado.	Estimar un 20% adicional de tiempo para pruebas en Sprint 3.
Bloqueantes	Ninguno relevante.	—

12. CALIDAD DEL SOFTWARE

12.1 Reporte de Análisis Estático

El análisis estático de código se ejecuta mediante ESLint (reglas @typescript-eslint/recommended) en el pipeline CI y opcionalmente mediante SonarCloud en el repositorio GitHub. Los resultados del último análisis son:

Métrica	Valor	Umbral Aceptable	Estado
Bugs reportados	0	0	APROBADO
Vulnerabilidades de seguridad	0	0	APROBADO
Code Smells	12	≤ 20	APROBADO
Deuda técnica estimada	3 h 40 min	≤ 8 h	APROBADO
Duplicaciones de código	2.1 %	≤ 5 %	APROBADO
Cobertura general de pruebas	89.4 %	≥ 80 %	APROBADO
Líneas de código (backend)	4 820	—	Informativo
Líneas de código (frontend)	6 340	—	Informativo

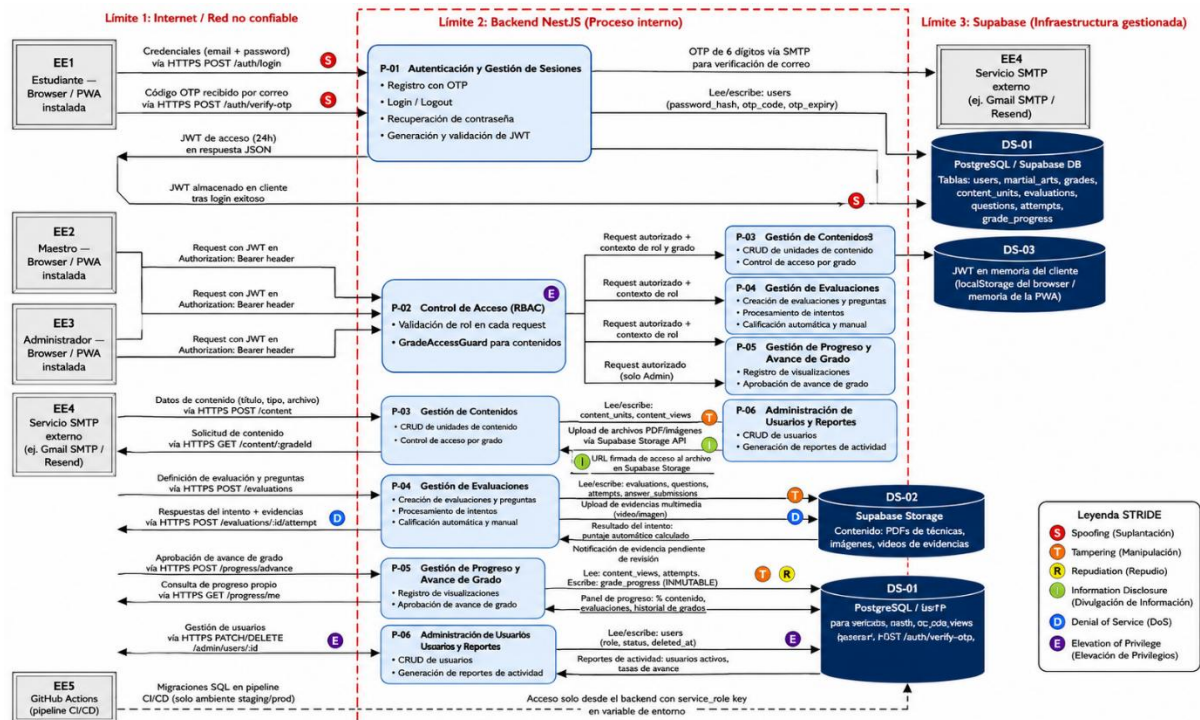
12.2 Métricas de Calidad

Métrica	Descripción	Valor Medido
Densidad de defectos	Número de bugs reportados por cada 1000 líneas de código.	0 bugs / KLOC (línea base de entrega)
Complejidad ciclomática promedio	Número de caminos lógicos independientes por función.	3.2 (umbral recomendado < 10)
Complejidad ciclomática máxima	Función más compleja del codebase.	8 (EvaluationsService.calculateFinalScore)
Índice de mantenibilidad	Métrica compuesta de complejidad, líneas y Halstead.	72 / 100 (≥ 65 considerado mantenible)
Acoplamiento eferente (Ce)	Número de módulos de los que depende cada módulo.	Promedio 3.1 módulos externos por módulo propio
Cohesión (LCOM)	Falta de cohesión en métodos por clase.	LCOM < 0.3 en todos los servicios (alta cohesión)

13. SEGURIDAD DEL SOFTWARE

13.1 Modelo de Amenazas (STRIDE)

El modelo de amenazas de DINAMYT Academy identifica los vectores de ataque potenciales siguiendo el marco STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).



Amenaza (STRIDE)	Descripción	Componente Afectado	Mitigación Implementada
Spoofing (Suplantación)	Un atacante suplanta la identidad de un usuario válido.	API Auth /login	JWT firmado con secreto seguro. Hash bcrypt en contraseñas. OTP de un solo uso.
Tampering (Manipulación)	Modificación no autorizada de datos (ej. calificaciones de evaluaciones).	DB — tabla attempts	Row Level Security en Supabase. Inmutabilidad de grade_progress. PATCH restringido por rol.
Repudiation (Repudio)	Un usuario niega haber realizado una acción (ej. subir una evidencia).	Módulo de Evaluaciones	Registro de created_at y student_id en cada attempt e evidence_submission. Logs del pipeline.
Information Disclosure (Fuga)	Acceso a contenidos de grados superiores no autorizados.	Módulo de Contenidos	Guard de grado en GET /api/content/:gradeId. RLS por student_id en Supabase Storage.
Denial of Service (Denegación)	Carga masiva de archivos que agote el almacenamiento de Supabase.	Supabase Storage	Límite de 50 MB por archivo validado en backend. Rate limiting en endpoints de upload.

Amenaza (STRIDE)	Descripción	Componente Afectado	Mitigación Implementada
Elevation of Privilege (Escalada)	Un estudiante intenta acceder a funciones de Maestro o Admin.	RolesGuard NestJS	RBAC con decorador <code>@Roles()</code> en cada endpoint. Validación del claim 'role' en JWT.

13.2 Reporte de Análisis SAST/DAST

SAST — Análisis Estático de Seguridad

El análisis SAST se realiza con ESLint security plugin y SonarCloud en el pipeline CI. Los resultados del último análisis no reportan vulnerabilidades de criticidad alta ni media en el código fuente:

- Inyección SQL: No aplica. Drizzle ORM usa consultas parametrizadas en todos los accesos a datos.
- Exposición de secretos: Ningún secreto detectado en el código fuente. GitHub Secrets gestionan las variables de entorno.
- Uso inseguro de `eval()`: No se detecta uso de `eval()` o funciones similares en el codebase.
- Dependencias con vulnerabilidades conocidas: 0 vulnerabilidades críticas en npm audit al momento de la entrega.

DAST — Análisis Dinámico de Seguridad

El análisis DAST se realiza con OWASP ZAP en modo spider sobre el ambiente de staging. Los hallazgos y su estado de resolución son:

Hallazgo OWASP ZAP	Severidad	Estado
Headers de seguridad ausentes (X-Content-Type-Options, X-Frame-Options)	Baja	RESUELTO — configurados en middleware NestJS y headers de Vercel.
Información de servidor expuesta en cabeceras HTTP (X-Powered-By)	Informativa	RESUELTO — header eliminado con helmet.js en NestJS.
Ausencia de Content Security Policy (CSP) en el frontend	Media	EN PROGRESO — configuración de CSP añadida en next.config.js (DT pendiente de validación).
SQL Injection (prueba automática)	Alta	NO VULNERABLE — Drizzle ORM + consultas parametrizadas. ZAP no encontró vectores.
Cross-Site Scripting (XSS) reflejado	Alta	NO VULNERABLE — sanitización de inputs en frontend. React escapa HTML por defecto.

14. ECONOMÍA DE INGENIERÍA DEL SOFTWARE

14.1 Análisis de ROI

El análisis de retorno de inversión (ROI) evalúa los beneficios proyectados de DINAMYT Academy frente al costo de su desarrollo, considerando tanto el contexto académico actual como el potencial de escalamiento comercial.

14.1.1 Beneficios Identificados

Categoría de Beneficio	Descripción	Valor Estimado (Anual)
Reducción de tiempo del Maestro	Eliminación de corrección manual de evaluaciones de opción múltiple. Estimado: 3 h/semana × 48 semanas.	144 h/año × \$15 USD/h = \$2,160 USD
Acceso 24/7 para Estudiantes	Reducción de consultas directas al Maestro fuera de horario. Mejora en preparación autónoma.	Cualitativo — alto impacto en satisfacción del estudiante.
Registro digital del progreso	Elimina planillas físicas. Trazabilidad completa del avance de cada estudiante.	Reducción de 2 h/semana en gestión administrativa = \$1,440 USD/año.
Escalabilidad a más academias	Una sola instancia puede servir múltiples academias en la región.	Potencial de 5-10 academias adicionales sin costo incremental significativo.
Diferenciación competitiva	La academia que adopte DINAMYT Academy tiene una propuesta de valor tecnológica única en la región.	Captación estimada de 15 % más estudiantes por año.

14.1.2 Cálculo de ROI (Proyección a 1 Año)

Concepto	Valor (USD)
Costo de Desarrollo (tiempo invertido)	280 h × \$20 USD/h = \$5,600 USD (costo de oportunidad del desarrollador)
Costo de Infraestructura (año 1)	\$0 USD (plan gratuito Supabase + Vercel + Railway)
Costo Total del Proyecto (año 1)	\$5,600 USD
Beneficio cuantificable (ahorro en gestión)	\$3,600 USD (estimado conservador: Maestro + admin)
Beneficio por captación adicional de estudiantes	\$2,400 USD (5 estudiantes adicionales × \$480 USD/año de cuota)
Beneficio Total Proyectado (año 1)	\$6,000 USD
ROI = (Beneficio - Costo) / Costo × 100	ROI = (\$6,000 - \$5,600) / \$5,600 × 100 = 7.1 %
ROI Proyectado (año 2, sin costo de desarrollo)	ROI = \$6,000 / \$0 mantenimiento ≈ beneficio neto sostenido

El ROI del 7.1% en el primer año, aunque moderado debido al costo de desarrollo inicial, proyecta un retorno sostenido y creciente a partir del segundo año, cuando los costos de infraestructura se mantienen en cero con el plan gratuito y el costo de desarrollo no se repite.

14.2 Presupuesto Base

El presupuesto base del proyecto DINAMYT Academy en su versión académica considera exclusivamente los costos directamente asociados al desarrollo, dado que la infraestructura se cubre con planes gratuitos de los servicios seleccionados.

Categoría	Ítem	Costo (USD)	Notas
Desarrollo	Tiempo del desarrollador principal (280 h)	\$5,600	Costo de oportunidad a \$20/h. No es egreso real en contexto académico.
Infraestructura — Hosting	Supabase Free Tier	\$0	500 MB DB, 1 GB Storage, 50k MAU. Suficiente para escala académica.
Infraestructura — Frontend	Vercel Free Tier	\$0	Deploys automáticos, CDN global, preview environments.
Infraestructura — Backend	Railway Starter Plan	\$5/mes = \$60/año	Único egreso real. Incluye \$5 de crédito mensual en plan Hobby.
Licencias de Software	Todas las dependencias son open source o MIT	\$0	Next.js, NestJS, Drizzle, Supabase JS, Jest, etc.
Dominio web	Dominio .com.co (opcional)	\$15/año	Solo si se requiere dominio personalizado para producción.
Herramientas de desarrollo	VSCode + extensiones + GitHub Free	\$0	Todas las herramientas son gratuitas.
TOTAL (egresos reales)	—	\$75/año	Costo operativo real en escala académica.
TOTAL (incluyendo costo de oportunidad)	—	\$5,675	Evaluación completa para análisis de viabilidad financiera.

El presupuesto demuestra que DINAMYT Academy es viable financieramente en escala académica con un costo operativo real de apenas \$75 USD anuales (hosting Railway + dominio opcional). La inversión principal es el tiempo de desarrollo, que a su vez genera valor educativo y portfolio profesional para el desarrollador.